چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۲۱

سیستم build و Gradle — بخش ۱: چرخهٔ build، Kotlin DSL، version catalog و build cache



اسماعیل مرادی

@esmaeeil-moradi700

چرا سیستم build در ۲۰۲۶ مهم است؟

هر تغییری که می‌دهید منتظر Gradle می‌ماند. در یک اپ ۱۰۰ ماژولی، build خودش یک محصول است — و مصاحبه‌گرها با آن کسانی را که فقط از Android Studio *استفاده* می‌کنند، از کسانی که می‌توانند *مالک* یک codebase باشند جدا می‌کنند.

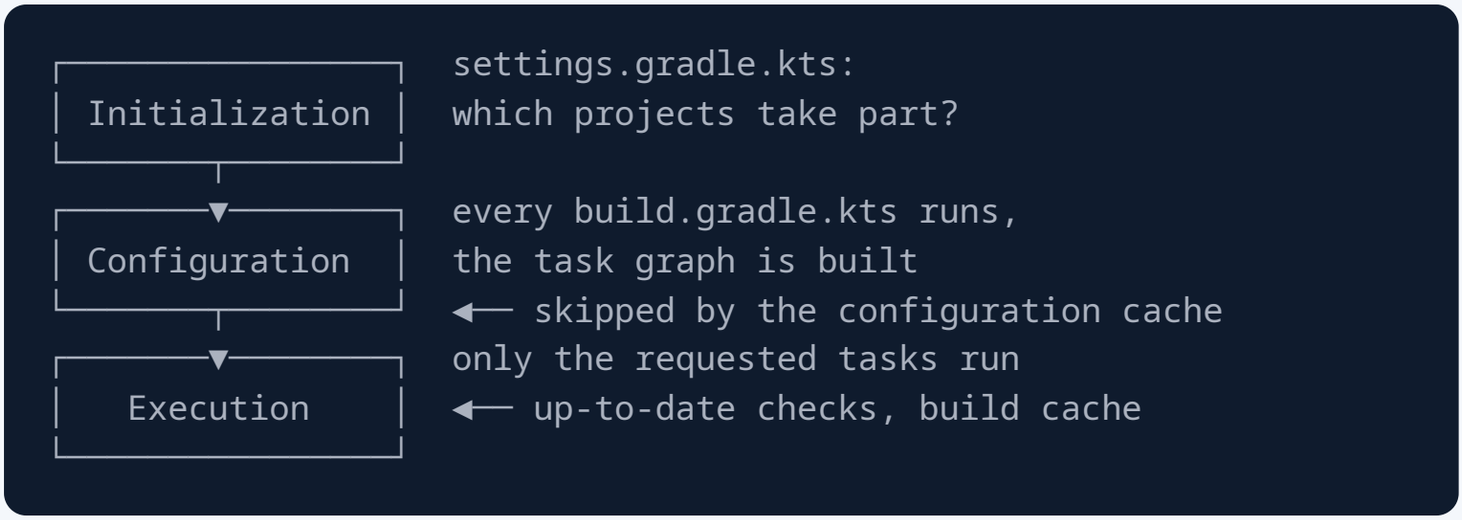
- زمان build یعنی سرعت تیم: هزینهٔ sync، build افزایشی و CI را هر مهندس، هر روز می‌پردازد
- خط پایهٔ مدرن: **Kotlin DSL**، **version catalog** و **convention plugin** در `build-logic`
- **configuration cache** حالت اجرای ترجیحی Gradle 9 است؛ **build cache** هنوز باید دستی روشن شود
- kapt در حالت نگهداری است و با Kotlin داخلی AGP 9 کار نمی‌کند — پیش‌فرض امروز **KSP** است
- بخش ۲ (روز ۲۲): وابستگی‌ها، variantها، قواعد R8، ارتقا و taskهای سفارشی

سطح	سؤال‌ها
جونیور	از سورس تا APK و AAB، فایل‌های Gradle، چرخهٔ build
مید	Kotlin DSL و سطح‌های SDK، version catalog، مقایسهٔ KSP و kapt، build cache
سینیور	configuration cache، convention pluginها، تنظیم `gradle.properties`
آرکیتکت	منطق build در مقیاس بزرگ، buildهای تکرارپذیر، مدرن‌سازی یک build قدیمی



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: سه فاز، دو cache



- **Initialization** فایل `settings.gradle.kts` را می‌خواند و تصمیم می‌گیرد کدام پروژه‌ها در این build حضور دارند
- **Configuration** اسکریپت‌های build همان پروژه‌ها را اجرا می‌کند و گراف taskها را می‌سازد — در هر build، مگر اینکه configuration cache دوباره استفاده شود
- **Execution** فقط taskهای لازم برای درخواست شما را اجرا می‌کند و taskهایی را که ورودی‌شان عوض نشده رد می‌کند

دو CACHE، دو فاز

configuration cache فاز **configuration** را رد می‌کند. build cache **اجرای taskها** را. مصاحبه‌گرها دوست دارند ببینند این تفاوت را می‌دانید.



اسماعیل مرادی
@esmaeeil-moradi700

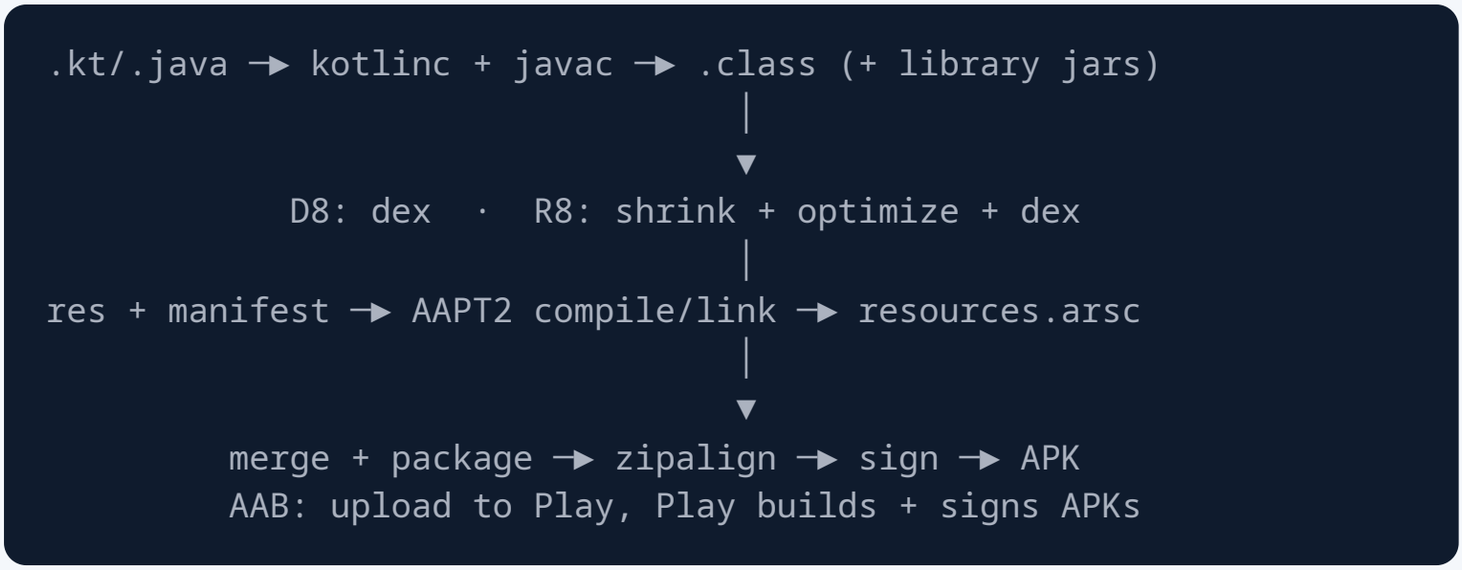
جونیور

از سورس تا APK و AAB، فایل‌های Gradle، چرخهٔ build



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. از سورس‌کد تا اپ قابل نصب



- **Gradle** فرایند build را اجرا می‌کند؛ پلاگین اندروید یا همان **AGP** taskهای مخصوص اندروید را اضافه می‌کند
- کد Kotlin و Java به bytecode ماشین مجازی جاوا کامپایل می‌شود؛ **D8** آن را به DEX برای runtime اندروید تبدیل می‌کند
- **AAPT2** منابع را کامپایل می‌کند و همراه manifest به `resources.arsc` و کلاس `R` لینک می‌کند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. جزئیاتی که مصاحبه‌گر می‌پرسد

- کلاس `R` مستقیماً به شکل bytecode ساخته می‌شود — فایل `R.java` برای ویرایش وجود ندارد. از AGP 8.0، کلاس‌های `R` **non-transitive** هستند: `R` هر ماژول فقط منابع خود همان ماژول را دارد.
- D8 همچنین APIهای جدیدتر جاوا را برای دستگاه‌های قدیمی **desugar** می‌کند. با `isMinifyEnabled = true`، **R8** جای آن را می‌گیرد و shrink، بهینه‌سازی، مبهم‌سازی و dex را در یک مرحله انجام می‌دهد.
- Google Play برای اپ‌های جدید **AAB** می‌خواهد. با Play App Signing، شما bundle را با *upload key* امضا می‌کنید؛ Play فایل‌های APK را می‌سازد و با کلید امضای اپ امضا می‌کند.

تلهٔ مصاحبه

«*اول APK را امضا کن، بعد zipalign.*» — برعکس است. با `apksigner`، zipalign **قبل از** امضا اجرا می‌شود: هر تغییری در فایل بعد از امضا، امضای نسخهٔ ۲ به بالا را باطل می‌کند. AGP هر دو را به ترتیب درست برایتان انجام می‌دهد.

سؤال تکمیلی

«*zipalign دقیقاً چه می‌کند؟*» — دادهٔ فشرده‌نشده را هم‌تراز می‌کند تا سیستم بتواند آن را مستقیم از داخل APK در حافظه map کند. کتابخانه‌های native فشرده‌نشده باید با اندازهٔ page هم‌تراز باشند — برای همین هم‌ترازی ۱۶ کیلوبایتی برای اپ‌های دارای کد native مهم است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. فایل‌های Gradle و کاربرد هر کدام

فایل	کاربرد
`settings.gradle.kts`	کدام ماژول‌ها وجود دارند، مخزن‌های plugin و وابستگی، buildهای includeشده
`build.gradle.kts` (ریشه)	اعلام pluginها با `apply false` — و در حالت ایده‌آل هیچ کار دیگری
`app/build.gradle.kts`	اعمال pluginها، تنظیمات `android {}`، وابستگی‌ها
`gradle/libs.versions.toml`	همان version catalog: کتابخانه‌ها، pluginها، نسخه‌ها
`gradle.properties`	پرچم‌های Gradle، JVM، Kotlin و AGP
`gradle/wrapper/gradle-wrapper.properties`	نسخهٔ دقیق Gradle که همه از طریق `./gradlew` می‌گیرند

تلهٔ مصاحبه

«فایل build ریشه بلوک‌های `buildscript {}` و `allprojects {}` را نگه می‌دارد.» — این چیدمان قدیمی است. امروز pluginها در بلوک `plugins {}` با aliasهای catalog اعلام می‌شوند و مخزن‌ها فقط در یک جا تعریف می‌شوند: `settings.gradle.kts`.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. یک `settings.gradle.kts` مدرن

```
pluginManagement {
    includeBuild("build-logic")
    repositories {
        google()
        mavenCentral()
        gradlePluginPortal()
    }
}
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}
rootProject.name = "Shop"
include(":app", ":core:data", ":feature:cart")
```

- با `FAIL_ON_PROJECT_REPOS`، اگر ماژولی مخزن اختصاصی خودش را تعریف کند build شکست می‌خورد — یک جا برای ممیزی اینکه artifactها از کجا می‌آیند
- `includeBuild("build-logic")` convention pluginهای شما را (سؤال ۹) با ID در دسترس می‌گذارد



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. چرخهٔ build در کد

```kotlin
// settings.gradle.kts — INITIALIZATION
include(":app", ":core:data")

// app/build.gradle.kts — CONFIGURATION (every build)
println("Configuring :app")

tasks.register("hello") {
    // Configuration action: runs only if 'hello' is needed
    group = "demo"
    doLast {
        // EXECUTION: runs only when 'hello' executes
        println("Hello from the execution phase")
    }
}
```

- `./gradlew hello` هر دو خط را چاپ می‌کند؛ `./gradlew help` هم باز «*Configuring :app*» را چاپ می‌کند
- فقط actionهای task — یعنی `doFirst`، `doLast` و `@TaskAction` — در فاز execution اجرا می‌شوند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. چیزهایی که مصاحبه‌گر چک می‌کند

تلهٔ مصاحبه

«*کد داخل* `tasks.register { }` *وقتی اجرا می‌شود که task اجرا شود.*» — نه. آن بلوک task را **پیکربندی** می‌کند. فقط actionهای آن (`doFirst` و `doLast`) در زمان execution اجرا می‌شوند.

تلهٔ مصاحبه

انجام کار واقعی در فاز configuration — خواندن فایل، صدا زدن `git`، درخواست شبکه — **هر** build و هر sync در Android Studio را کند می‌کند، حتی وقتی هیچ‌کس آن task را اجرا نمی‌کند.

سؤال تکمیلی

«*چرا Gradle sync کند است؟*» — بیشترِ زمان sync صرف configuration و resolve کردن وابستگی‌ها می‌شود. هر کاری که در زمان configuration انجام شود، مستقیم به زمان sync اضافه می‌شود.



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

Kotlin DSL و سطح‌های SDK، version catalog، مقایسهٔ KSP و kapt، build cache



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. فایل build یک ماژول با Kotlin DSL

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.compose)
}

android {
    namespace = "com.example.shop"
    compileSdk = 36
    defaultConfig {
        applicationId = "com.example.shop"
        minSdk = 24
        targetSdk = 36
    }
}

dependencies {
    implementation(libs.androidx.core.ktx)
}
```

- Kotlin DSL پیش‌فرض پروژه‌های جدید Android Studio است: بررسی در زمان کامپایل، تکمیل خودکار و رفتن به تعریف DSL در IDE
- با **AGP 9**، پشتیبانی از Kotlin داخلی شده است — دیگر plugin `org.jetbrains.kotlin.android` لازم نیست. plugin کامپایلر Compose هنوز جداگانه اعمال می‌شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. سطح‌های SDK — و یک تلهٔ Kotlin DSL

تنظیم	معنی
`compileSdk`	با کدام APIهای اندروید می‌توانید **کامپایل** کنید. رفتار زمان اجرا را تغییر نمی‌دهد
`minSdk`	پایین‌ترین نسخهٔ اندرویدی که اپ روی آن نصب می‌شود؛ Lint فراخوانی APIهای جدیدتر را گزارش می‌کند
`targetSdk`	**تغییرات رفتاری** کدام نسخه را می‌پذیرید. Play یک نسخهٔ جدید را الزامی می‌کند

تلهٔ مصاحبه

«*targetSdk را همیشه روی آخرین نسخه بگذار.*» — بالا بردنش یک **مهاجرت** است، نه عوض کردن یک عدد: target کردن API 35 حالت edge-to-edge را اجباری می‌کند؛ با target کردن API 36 دیگر `onBackPressed()` صدا زده نمی‌شود. اول تغییرات رفتاری را بخوانید و تست کنید.

تلهٔ مصاحبه

accessorهای type-safe — مثل `android {}` و `implementation(…)` — فقط برای pluginهایی ساخته می‌شوند که در بلوک `plugins {}` اعمال شده‌اند. با `apply(plugin = "…")` اسکریپت با خطای «*unresolved reference*» شکست می‌خورد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. version catalog

```
# gradle/libs.versions.toml
[versions]
agp = "9.1.0"
ksp = "2.3.11"
room = "2.7.2"

[libraries]
room-runtime = { module = "androidx.room:room-runtime", version.ref = "room" }
room-compiler = { module = "androidx.room:room-compiler", version.ref = "room" }
junit = "junit:junit:4.13.2"

[plugins]
android-library = { id = "com.android.library", version.ref = "agp" }
ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }
```

- در alias، کاراکترهای `-` و `_` و `.` در accessor به `.` تبدیل می‌شوند: `room-runtime` در کد می‌شود `libs.room.runtime`
- نسخهٔ pluginها یک بار اعلام می‌شود؛ ماژول‌ها آن‌ها را با alias و بدون نسخه اعمال می‌کنند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. استفاده از catalog

```
plugins {
    alias(libs.plugins.android.library)
    alias(libs.plugins.ksp)
}

dependencies {
    implementation(libs.room.runtime)
    ksp(libs.room.compiler)
    testImplementation(libs.junit)
}
```

- یک فایل برای review، و یک فایل برای باتهای به‌روزرسانی مثل Renovate
- جای ثابت‌های قدیمی `buildSrc/Dependencies.kt` و نسخه‌هایی را که در `gradle.properties` نگه داشته می‌شدند می‌گیرد

تلهٔ مصاحبه

«*catalog نسخه‌ها را برای کل اپ قفل می‌کند.*» — نه. catalog فهرستی از اسم‌ها و نسخه‌های **درخواستی** است. conflict resolution همچنان می‌تواند نسخهٔ بالاتری را که به‌صورت transitive آمده انتخاب کند. هم‌تراز کردن و اجبار نسخه‌ها کار platformها، BOMها و constraintهاست — روز ۲۲.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. مقایسهٔ KSP و kapt

	kapt	KSP
نحوهٔ کار	از کد Kotlin شما stub جاوا می‌سازد، بعد annotation processorهای جاوا را اجرا می‌کند	symbolهای Kotlin را مستقیم می‌خواند — بدون stub
سرعت	ساخت stub و اجرای javac در هر تغییر	طبق مستندات رسمی «تا ۲ برابر سریع‌تر»
درک Kotlin	نمای جاوایی کد شما را می‌بیند	nullability و ساختارهای Kotlin را می‌فهمد
وضعیت	در حالت نگهداری؛ با Kotlin `kotlin-kapt` داخلی AGP 9 ناسازگار است	استاندارد فعلی: Room، Dagger و Hilt، Moshi، Glide
چندسکویی	فقط JVM	در پروژه‌های KMP کار می‌کند

تلهٔ مصاحبه

«*KSP همان kapt سریع‌تر است.*» — یک API متفاوت است. annotation processorهای جاوا روی KSP اجرا نمی‌شوند؛ هر کتابخانه باید processor مخصوص KSP ارائه کند. Data Binding هرگز این کار را نمی‌کند — آن را در ماژول‌های جدا نگه دارید یا به View Binding مهاجرت کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. مهاجرت، ماژول به ماژول

```
plugins {
    alias(libs.plugins.android.library)
    alias(libs.plugins.ksp)
    alias(libs.plugins.hilt)
}

dependencies {
    implementation(libs.room.runtime)
    ksp(libs.room.compiler)       // was: kapt(…)
    implementation(libs.hilt.android)
    ksp(libs.hilt.compiler)
}
```

- بیشتر کتابخانه‌ها processor مربوط به KSP را در همان artifact ارائه می‌کنند: کافی است `kapt(` را به `ksp(` تغییر دهید
- در AGP 9، ماژولی که هنوز نمی‌تواند kapt را کنار بگذارد، به‌طور موقت از plugin `com.android.legacy-kapt` استفاده می‌کند
- از KSP 2.3.0 به بعد، نسخهٔ KSP دیگر به نسخهٔ Kotlin گره نخورده است

تلهٔ مصاحبه

نگه داشتن kapt و KSP کنار هم در یک ماژول «چون کار می‌کند» — تا وقتی **حتی یک** processor مربوط به kapt باقی مانده، ماژول همچنان stub می‌سازد. برد اصلی فقط وقتی به دست می‌آید که آخرین استفاده از kapt از ماژول حذف شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. بررسی up-to-date در برابر build cache

نتیجهٔ task	معنی
`UP-TO-DATE`	ورودی‌ها و خروجی‌ها از آخرین اجرا **در همین workspace** تغییر نکرده‌اند
`FROM-CACHE`	خروجی‌ها از build cache برگردانده شده‌اند — قبلاً، در branch یا ماشین دیگری ساخته شده‌اند
`NO-SOURCE`	چیزی برای پردازش وجود ندارد
(بدون برچسب)	task واقعاً اجرا شده است

- هر دو به **ورودی‌ها و خروجی‌های اعلام‌شده** وابسته‌اند. کلید cache یک hash از پیاده‌سازی task و همهٔ ورودی‌های آن است.
- build cache **به‌طور پیش‌فرض خاموش است**؛ با `org.gradle.caching=true` یا پرچم `--build-cache` روشنش کنید
- cache محلی در `~/.gradle/caches/build-cache-1` است؛ cache ریموت مشترک موضوع روز ۲۲ است



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. build cache در عمل

```
$ ./gradlew clean :app:assembleDebug
> Task :core:data:compileDebugKotlin FROM-CACHE
> Task :feature:cart:compileDebugKotlin FROM-CACHE
> Task :app:compileDebugKotlin
…
BUILD SUCCESSFUL
312 actionable tasks: 41 executed, 271 from cache
```

تلهٔ مصاحبه

«*build cache محلی به‌طور پیش‌فرض فعال است.*» — نیست (نسخهٔ پارسال همین چالش این را اشتباه گفته بود). بدون `org.gradle.caching=true` خروجی taskها هرگز ذخیره یا دوباره استفاده نمی‌شوند.

تلهٔ مصاحبه

«`clean` *کش را بی‌فایده می‌کند.*» — برعکس: بعد از `clean`، taskهای cacheable با وضعیت `FROM-CACHE` برمی‌گردند. چیزی که `clean` از بین می‌برد فقط وضعیت up-to-date در پوشه‌های `build/` است.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

configuration cache، convention pluginها، تنظیم gradle.properties



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. configuration cache

نتیجهٔ فاز configuration را ذخیره می‌کند — گراف taskها و ورودی‌های هر task — و تا وقتی چیزی که روی configuration اثر داشته تغییر نکند، دوباره از آن استفاده می‌کند: اسکریپت‌های build، `gradle.properties`، متغیرهای محیطی یا فایل‌هایی که در زمان configuration خوانده شده‌اند.

```
# gradle.properties
org.gradle.configuration-cache=true
# while migrating only: report problems instead of failing
org.gradle.configuration-cache.problems=warn
```

	configuration cache	build cache
چه چیزی را رد می‌کند	فاز configuration (اسکریپت‌ها، گراف taskها)	اجرای taskها
اشتراک بین ماشین‌ها	نه — محلیِ همان build	بله، با cache ریموت
وضعیت	پایدار از Gradle 8.1؛ حالت ترجیحی در Gradle 9، اما هنوز دستی	دستی

با روشن بودن configuration cache، taskها **داخل یک پروژه هم به‌صورت موازی** اجرا می‌شوند، نه فقط بین ماژول‌ها.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. چه چیزی خرابش می‌کند — و راه‌حل

```
// ❌ Uses Project at execution time
tasks.register("printVersion") {
    doLast { println(project.version) }
}

// ✅ Capture the value during configuration
tasks.register("printVersion") {
    val version = project.version.toString()
    doLast { println(version) }
}

// ✅ Read the environment through providers
val isCi = providers.environmentVariable("CI").isPresent
```

- ناسازگارهای دیگر: build listenerهایی مثل `gradle.buildFinished` و `{}` `TaskExecutionListener`، و pluginهایی که به‌روز نشده‌اند
- گزارش HTML در مسیر `build/reports/configuration-cache/` همهٔ مشکلات را فهرست می‌کند

تلهٔ مصاحبه

«با `problems=warn` *اجرا می‌کنیم، پس سازگاریم.*» — حالت warning فقط ابزار مهاجرت است. مشکلاتی را پنهان می‌کند که می‌توانند build نادرست تولید کنند؛ مستندات Gradle می‌گوید آن را به‌عنوان تنظیم دائمی نگه ندارید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. convention pluginها در `build-logic`

به‌جای کپی کردن یک بلوک `android {}` در ۴۰ ماژول، آن را یک بار به شکل plugin بنویسید و با ID اعمالش کنید.

```
build-logic/
├── settings.gradle.kts    reuses ../gradle/libs.versions.toml
└── convention/
    ├── build.gradle.kts   kotlin-dsl + gradlePlugin {}
    └── src/main/kotlin/AndroidLibraryConventionPlugin.kt
```

```kotlin
// build-logic/convention/build.gradle.kts
plugins { `kotlin-dsl` }

dependencies {
    compileOnly(libs.android.gradlePlugin)
}

gradlePlugin {
    plugins {
        register("androidLibrary") {
            id = "shop.android.library"
            implementationClass = "AndroidLibraryConventionPlugin"
        }
    }
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. خود plugin و نحوهٔ استفاده

```
class AndroidLibraryConventionPlugin : Plugin<Project> {
    override fun apply(target: Project) {
        with(target) {
            pluginManager.apply("com.android.library")
            extensions.configure<LibraryExtension> {
                compileSdk = 36
                defaultConfig.minSdk = 24
            }
            dependencies {
                val junit = libs.findLibrary("junit").get()
                add("testImplementation", junit)
            }
        }
    }
}

// core/data/build.gradle.kts
plugins { id("shop.android.library") }
```

- `LibraryExtension` همان نوع DSL جدید از `com.android.build.api.dsl` است — تنها نوعی که AGP 9 در اختیار می‌گذارد
- accessorهای تولیدشدهٔ `libs.x` داخل کلاس plugin وجود ندارند؛ در اینجا یک extension کوچک روی `VersionCatalogsExtension` است

تلهٔ مصاحبه

«*استفاده از `allprojects {}` در اسکریپت ریشه ساده‌تر است.*» — پیکربندی بین‌پروژه‌ای همهٔ ماژول‌ها را به ریشه گره می‌زند، منبع تنظیمات را پنهان می‌کند و جلوی Isolated Projects در Gradle را می‌گیرد. convention pluginها انتخابی‌اند.



اسماعیل مرادی

@esmaeeil-moradi700

سؤال ۱۰. تنظیم `gradle.properties`

```
org.gradle.jvmargs=-Xmx4g -XX:+UseParallelGC -Dfile.encoding=UTF-8
kotlin.daemon.jvmargs=-Xmx3g
org.gradle.parallel=true
org.gradle.caching=true
org.gradle.configuration-cache=true
```

ویژگی	اثر	پیش‌فرض
`org.gradle.jvmargs`	heap و GC دیمن Gradle، جایی که taskهای AGP اجرا می‌شوند	کم — تنظیمش کنید
`kotlin.daemon.jvmargs`	heap دیمن جداگانهٔ کامپایلر Kotlin	از Gradle به ارث می‌برد
`org.gradle.parallel`	اجرای موازی taskهای پروژه‌های مختلف	خاموش
`org.gradle.caching`	همان build cache	خاموش
`org.gradle.configuration-cache`	همان configuration cache	خاموش

مقادیر وابسته به ماشین (مثل اندازهٔ heap) را می‌توانید در `~/.gradle/gradle.properties` بگذارید؛ این فایل بر فایل پروژه اولویت دارد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. افسانه‌های سرعت build

تلهٔ مصاحبه

«*برای سریع‌تر شدن build از* `--offline` *استفاده کن.*» — با نسخه‌های ثابت، Gradle از قبل از cache محلی وابستگی‌ها استفاده می‌کند و سراغ شبکه نمی‌رود. چیزی که Gradle را مجبور به چک کردن شبکه می‌کند نسخه‌های **dynamic** مثل `1.+` و نسخه‌های `SNAPSHOT` است — آن‌ها را حذف کنید.

تلهٔ مصاحبه

«*heap بیشتر همیشه سریع‌تر است.*» — heap بیش از نیاز build سرعت را بالا نمی‌برد و حافظه را از Android Studio و emulator می‌گیرد. زمان GC را قبل و بعد از تغییر در یک build scan بررسی کنید — روز ۲۲.

چیزی که واقعاً فرق ایجاد می‌کند

configuration cache، build cache، حذف kapt، APIهای lazy برای taskها، و گراف ماژولی با دامنهٔ recompile کوچک (ترجیح `implementation` به `api` — روز ۲۲). پرچم‌ها فقط چیزی را آزاد می‌کنند که طراحی build اجازه‌اش را می‌دهد.



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

منطق build در مقیاس بزرگ، buildهای تکرارپذیر، مدرن‌سازی یک build قدیمی



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. منطق build به‌عنوان یک محصول

```
shop.jvm.library           pure Kotlin modules (domain)
shop.android.library       compileSdk, minSdk, Lint, tests
shop.android.compose       Compose compiler plugin + BOM
shop.android.feature       library + compose + DI + :core:ui
shop.android.application   signing, R8, versioning
```

- **pluginها را ترکیب کنید، نه ارث‌بری:** pluginهای فیچر، pluginهای library و compose را اعمال می‌کند؛ هر دغدغه دقیقاً در یک جا تعریف می‌شود
- فایل build یک ماژول فقط می‌گوید **چیست** (IDهای plugin) و **به چه نیاز دارد** (وابستگی‌ها) — نه چیز دیگر
- pluginها را مثل کد production تست کنید، با Gradle TestKit و `GradleRunner`

تلهٔ مصاحبه

«*buildSrc منسوخ شده — باید build-logic استفاده کنید.*» — هیچ‌کدام منسوخ نیستند؛ هر دو می‌توانند convention plugin داشته باشند. یک build جداگانهٔ includeشده به اسم `build-logic` (چیدمان Now in Android) صریح‌تر است: هیچ چیزی به‌طور ضمنی روی classpath همهٔ اسکریپت‌ها نمی‌نشیند — ماژول‌ها با ID پلاگین آن را انتخاب می‌کنند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. configuration را lazy نگه دارید

```
// ❌ Eager: creates and configures every Test task now
tasks.withType<Test> {
    maxHeapSize = "1g"
}

// ✅ Lazy: configures only tasks that end up in the graph
tasks.withType<Test>().configureEach {
    maxHeapSize = "1g"
}

// ✅ Lazy lookup by name
tasks.named<Test>("testDebugUnitTest") {
    useJUnitPlatform()
}
```

- زمان configuration با هر ماژول بیشتر می‌شود — کد plugin برای هر پروژه یک بار اجرا می‌شود، در هر build بدون hit در configuration cache، و در هر sync
- دست بردن در پروژه‌های دیگر (`project(":x").extensions...`) و ترفندهای ترتیبی با `afterEvaluate` ممنوع — مقادیر را با `Provider`های lazy به هم وصل کنید

تلهٔ مصاحبه

«`tasks.withType<T> { }` *همان API تنبل است.*» — overload با lambda **همهٔ** taskهای آن نوع را می‌سازد. فقط `withType<T>().configureEach { }` کار را به تعویق می‌اندازد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. buildهای تکرارپذیر: ورودی یکسان، نتیجهٔ یکسان، روی هر ماشینی

ریسک	کنترل
نسخه‌های مختلف Gradle	wrapper را commit کنید، `distributionSha256Sum` را تنظیم کنید، jar مربوط به wrapper را در CI اعتبارسنجی کنید
JDKهای مختلف	تعریف Java toolchain با `kotlin { jvmToolchain(17) }` — کامپایل دیگر به `JAVA_HOME` وابسته نیست
artifactهای دست‌کاری‌شده یا جابه‌جاشده	تأیید وابستگی‌ها با فایل `gradle/verification-metadata.xml`
نسخه‌های متحرک	بدون نسخهٔ dynamic یا `SNAPSHOT`؛ اگر مجبور به بازه هستید، dependency locking
خروجی‌های غیرقطعی	هیچ timestamp یا مسیر ماشین در کد یا resourceهای تولیدشده

تلهٔ مصاحبه

«*روی ماشین من build می‌شود، پس JDK مشکلی ندارد.*» — آن JDK که Gradle را **اجرا** می‌کند (تنظیم Gradle JDK در Android Studio یا `JAVA_HOME`) و toolchainی که کد شما را **کامپایل** می‌کند دو تنظیم جدا هستند. Gradle 9 و AGP 9 هر دو برای اجرا به JDK 17 یا بالاتر نیاز دارند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. در عمل

```
# Pin Gradle and its checksum for everyone
./gradlew wrapper --gradle-version 9.1.0 \
    --gradle-distribution-sha256-sum <sha256>

# Record checksums of every dependency and plugin
./gradlew --write-verification-metadata sha256 help
```

```
// ❌ A new value on every build
buildConfigField(
    "long", "BUILD_TIME", "${System.currentTimeMillis()}L"
)
```

تلهٔ مصاحبه

یک timestamp یا git SHA در `BuildConfig` نسخهٔ **debug**، در هر build یک ورودی را عوض می‌کند: ماژول دوباره کامپایل می‌شود و همهٔ ماژول‌های پایین‌دستی cache را از دست می‌دهند. این مقادیر را فقط در buildهای release یا CI تزریق کنید، یا در زمان اجرا بخوانید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. مدرن‌سازی یک build قدیمی و کند

سناریو: sync کند است، CI کند است، و ۶۰ ماژول هر کدام فایل build کپی‌شدهٔ خودشان را دارند. به چه ترتیبی درستش می‌کنید؟

#	قدم	چرا این‌جا
۱	اندازه‌گیری خط پایه با build scan و Build Analyzer	بدون عدد نمی‌توانید بهبود را ثابت کنید — روز ۲۲
۲	ارتقای Gradle، AGP و Kotlin	رفع مشکلات کارایی؛ پیش‌نیاز بقیهٔ قدم‌ها
۳	اول version catalog، بعد convention pluginها	هر تغییر بعدی، تغییر در یک فایل می‌شود
۴	اول build cache، بعد configuration cache — ابتدا حالت warn، بعد رفع مشکلات، بعد حالت سخت‌گیرانه	بزرگ‌ترین بردها برای build محلی و CI
۵	مهاجرت ماژول‌به‌ماژول از kapt به KSP	حذف ساخت stub
۶	شکستن ماژول‌های داغ؛ ترجیح `implementation` به `api`	دامنهٔ recompile کوچک‌تر — روزهای ۲۲ و ۲۵



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. اجرای مهاجرت

تلهٔ مصاحبه

PR انفجاری — مهاجرت از Groovy به Kotlin DSL، از buildSrc به build-logic، از kapt به KSP و ارتقای major در AGP، همه با هم. نه می‌شود reviewاش کرد، نه bisect، و نه فهمید کدام تغییر باعث regression شد. هر PR یک تغییر، و هر تغییر اندازه‌گیری‌شده.

تلهٔ مصاحبه

«*تبدیل به Kotlin DSL، build را سریع‌تر می‌کند.*» — به‌تنهایی نه. Kotlin DSL برای درستی و نگهداری‌پذیری است؛ سرعت از caching، configuration تنبل و گراف ماژول‌ها می‌آید.

سؤال تکمیلی

«*بعد از پروژه چطور سریع نگهش می‌دارید؟*» — زمان build را مثل یک بودجه ببینید: زمان configuration و نرخ hit در cache را در CI دنبال کنید و برای regressionها هشدار بگذارید (روزهای ۲۲ و ۲۸).

جواب سینیور در یک جمله

«build سریع طراحی می‌شود، نه تنظیم: configuration تنبل، taskهای cacheable، ماژول‌های کوچک، و یک جا برای منطق build.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۲۱ — خلاصهٔ یک‌صفحه‌ای

- مسیر build: کامپایل با kotlinc و javac، ساخت DEX با D8 یا R8، منابع با AAPT2؛ بعد zipalign و **سپس** امضا. Play یک AAB می‌گیرد.
- سه فاز: initialization (کدام پروژه‌ها)، configuration (اسکریپت‌ها و گراف taskها)، execution (فقط taskهای لازم).
- هیچ کار واقعی در زمان configuration نکنید؛ کار task جایش در `doLast` است.
- Kotlin DSL، بلوک `plugins {}` و version catalog خط پایه‌اند. catalog نسخه‌ها را **نام‌گذاری** می‌کند — اجبارشان نمی‌کند.
- `compileSdk` یعنی APIهایی که با آن‌ها کامپایل می‌کنید؛ `targetSdk` یعنی رفتاری که می‌پذیرید. بالا بردنش مهاجرت است.
- KSP به‌جای kapt؛ حتی یک processor باقی‌مانده از kapt هزینهٔ stub را نگه می‌دارد.
- وضعیت `UP-TO-DATE` با `FROM-CACHE` فرق دارد. build cache باید دستی روشن شود.
- configuration cache فاز configuration را رد می‌کند؛ build cache اجرا را.
- convention pluginها در `build-logic` و اعمال با ID. بدون `allprojects` `{}`؛ `configureEach` به‌جای APIهای eager.
- تکرارپذیری: wrapper با checksum، JDK toolchain، تأیید وابستگی‌ها، و بدون timestamp در ورودی‌ها.

روز ۲۲ در راه است: Gradle بخش ۲ — وابستگی‌ها، variantها، قواعد R8 و ارتقا.
این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700